Northeastern University

EECE5155: Wireless Sensor Networks and the Internet of Things
Laboratory Assignment Report 1

Authors:
- Riya Rawal
- Shashank Meghashyam
- Group number on Canvas: 3

Task 1

1. Experimental setup

Network Specifications:
physical (802.11ac, default channel), link (ad-hoc MAC), network (IPv4, 192.168.1.0/24), transport (UDP), application (echo server and 2 clients), pcap on Node 1.

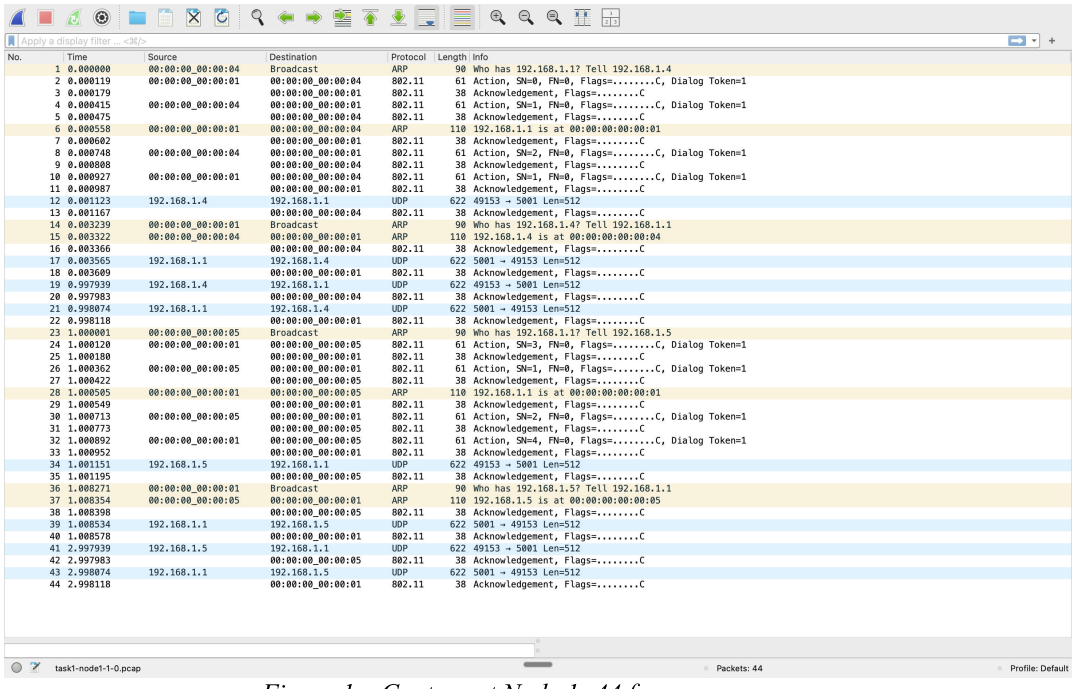
Code	Explanation
wifiNodes.Create(5)	No access points, all nodes are peers
wifi.SetStandard(WIFI_STANDARD_80211ac)	Following 802.11ac defaults
YansWifiChannelHelper::Default() and YansWifiPhyHelper	Setting channel mode
GridPositionAllocator (MinX 0, MinY 0, DeltaX 5, DeltaY 10, GridWidth 3)	Initial node positions, inside the walk area
RandomWalk2dMobilityModel with Rectangle(-90, 90, -90, 90)	Setting node mobility within specified area
address.SetBase("192.168.1.0", "255.255.255.0")	Puts nodes in subnet
Ipv4GlobalRoutingHelper::PopulateRoutingTables()	Ideal routers exchanging routing tables in the background
UdpEchoServerHelper echoServer(5001) on Get(0)	Server on Node 0, port 5001
Client on Get(3): Start(1.0), Interval(1.0) and MaxPackets(2)	Sends at 1s and 2s
Client on Get(4): Start(2.0), Interval(2.0) and MaxPackets(2)	Sends at 2s and 4s
PacketSize = 512	Tells packet size
SetPcapDataLinkType(DLT_IEEE802_11_RADIO) and EnablePcap("task1-node1", devices.Get(1))	Captures only on Node1. Header is needed so Wireshark can decode frames correctly.
mac.SetType("ns3::AdhocWifiMac")	Makes the network Adhoc

2. Results

- **Ans1**

Total frames= 44
Broadcast Frames = 4
Unicast Frames = 20
ACK Frames = 20

No, not all frames are acknowledged with an ACK frame. Only unicast frames are. Broadcast frames are not. An ACK frame only carries the receiver's address: it is a reply from one specific receiver to one specific sender. A broadcast has no single receiver, and if every node sent back an ACK, the ACKs would collide with each other. Broadcast frames are therefore sent once and are never acknowledged.
The 44 frames are: 8 ARP frames (4 broadcast requests and 4 unicast replies), 8 UDP frames (4 echo requests and 4 echo replies), 8 Action frames (Block Ack setup) and 20 ACK frames. The 20 unicast frames are therefore the 8 UDP frames, the 4 ARP replies and the 8 Action frames, and each of them is followed by an ACK.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:00:00_00:00:04	Broadcast	ARP	90	Who has 192.168.1.1? Tell 192.168.1.4
2	0.000119	00:00:00_00:00:01	00:00:00_00:00:04	802.11	61	Action, SN=0, FN=0, Flags=........C, Dialog Token=1
3	0.000179		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
4	0.000415	00:00:00_00:00:04	00:00:00_00:00:01	802.11	61	Action, SN=1, FN=0, Flags=........C, Dialog Token=1
5	0.000475		00:00:00_00:00:04	802.11	38	Acknowledgement, Flags=........C
6	0.000558	00:00:00_00:00:01	00:00:00_00:00:04	ARP	110	192.168.1.1 is at 00:00:00:00:00:01
7	0.000602		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
8	0.000748	00:00:00_00:00:04	00:00:00_00:00:01	802.11	61	Action, SN=2, FN=0, Flags=........C, Dialog Token=1
9	0.000808		00:00:00_00:00:04	802.11	38	Acknowledgement, Flags=........C
10	0.000927	00:00:00_00:00:01	00:00:00_00:00:04	802.11	61	Action, SN=1, FN=0, Flags=........C, Dialog Token=1
11	0.000987		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
12	0.001123	192.168.1.4	192.168.1.1	UDP	622	49153 → 5001 Len=512
13	0.001167		00:00:00_00:00:04	802.11	38	Acknowledgement, Flags=........C
14	0.003239	00:00:00_00:00:01	Broadcast	ARP	90	Who has 192.168.1.4? Tell 192.168.1.1
15	0.003322	00:00:00_00:00:04	00:00:00_00:00:01	ARP	110	192.168.1.4 is at 00:00:00:00:00:04
16	0.003366		00:00:00_00:00:04	802.11	38	Acknowledgement, Flags=........C
17	0.003565	192.168.1.1	192.168.1.4	UDP	622	5001 → 49153 Len=512
18	0.003609		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
19	0.997939	192.168.1.4	192.168.1.1	UDP	622	49153 → 5001 Len=512
20	0.997983		00:00:00_00:00:04	802.11	38	Acknowledgement, Flags=........C
21	0.998074	192.168.1.1	192.168.1.4	UDP	622	5001 → 49153 Len=512
22	0.998118		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
23	1.000001	00:00:00_00:00:05	Broadcast	ARP	90	Who has 192.168.1.1? Tell 192.168.1.5
24	1.000120	00:00:00_00:00:01	00:00:00_00:00:05	802.11	61	Action, SN=3, FN=0, Flags=........C, Dialog Token=1
25	1.000180		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
26	1.000362	00:00:00_00:00:05	00:00:00_00:00:01	802.11	61	Action, SN=1, FN=0, Flags=........C, Dialog Token=1
27	1.000422		00:00:00_00:00:05	802.11	38	Acknowledgement, Flags=........C
28	1.000505	00:00:00_00:00:01	00:00:00_00:00:05	ARP	110	192.168.1.1 is at 00:00:00:00:00:01
29	1.000549		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
30	1.000713	00:00:00_00:00:05	00:00:00_00:00:01	802.11	61	Action, SN=2, FN=0, Flags=........C, Dialog Token=1
31	1.000773		00:00:00_00:00:05	802.11	38	Acknowledgement, Flags=........C
32	1.000892	00:00:00_00:00:01	00:00:00_00:00:05	802.11	61	Action, SN=4, FN=0, Flags=........C, Dialog Token=1
33	1.000952		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
34	1.001151	192.168.1.5	192.168.1.1	UDP	622	49153 → 5001 Len=512
35	1.001195		00:00:00_00:00:05	802.11	38	Acknowledgement, Flags=........C
36	1.008271	00:00:00_00:00:01	Broadcast	ARP	90	Who has 192.168.1.5? Tell 192.168.1.1
37	1.008354	00:00:00_00:00:05	00:00:00_00:00:01	ARP	110	192.168.1.5 is at 00:00:00:00:00:05
38	1.008398		00:00:00_00:00:05	802.11	38	Acknowledgement, Flags=........C
39	1.008534	192.168.1.1	192.168.1.5	UDP	622	5001 → 49153 Len=512
40	1.008578		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C
41	2.997939	192.168.1.5	192.168.1.1	UDP	622	49153 → 5001 Len=512
42	2.997983		00:00:00_00:00:05	802.11	38	Acknowledgement, Flags=........C
43	2.998074	192.168.1.1	192.168.1.5	UDP	622	5001 → 49153 Len=512
44	2.998118		00:00:00_00:00:01	802.11	38	Acknowledgement, Flags=........C

Figure 1a. Capture at Node 1: 44 frames.

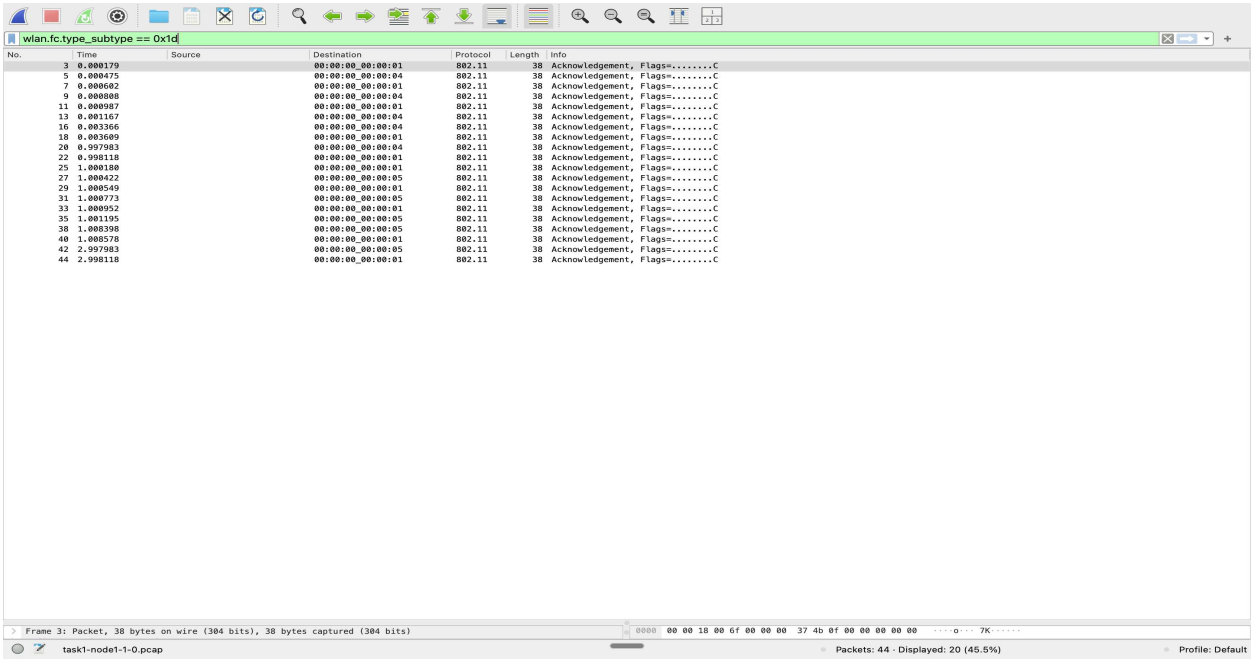


Figure 1b. Capture with filter wlan.fc.type_subtype == 0x1d: status bar shows 20 ACK frames displayed.

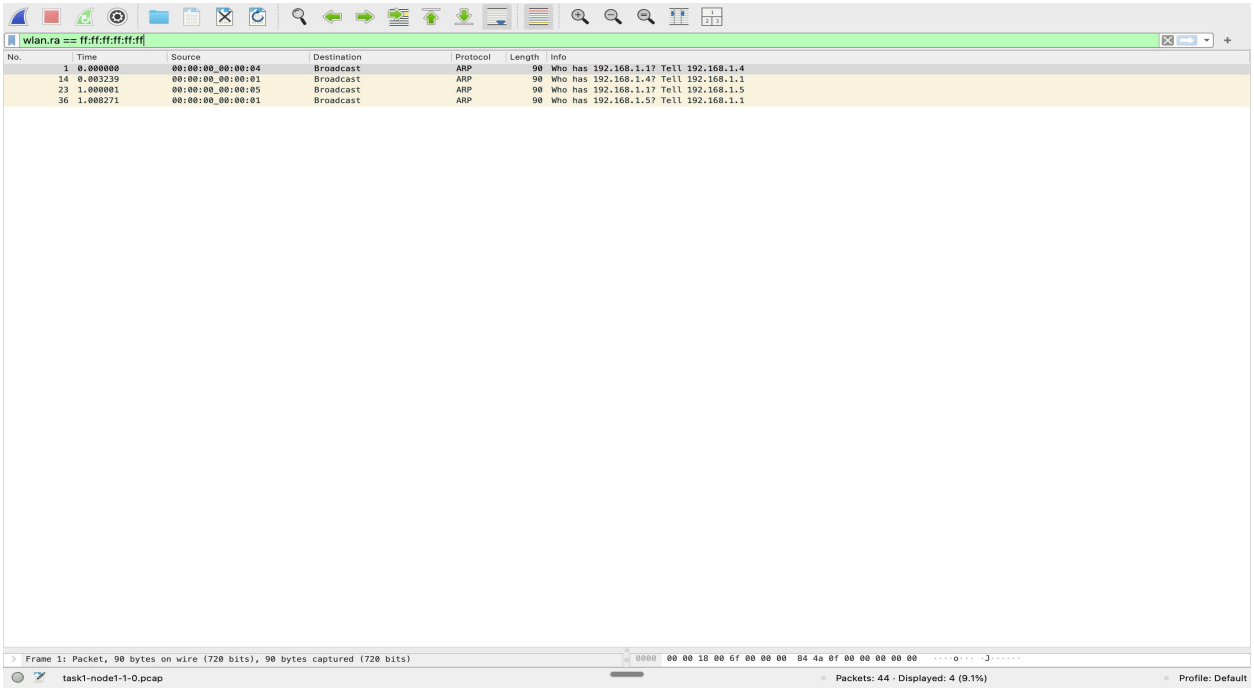


Figure 1c. Capture with filter wlan.ra == ff:ff:ff:ff:ff:ff: status bar shows 4 broadcast frames displayed.

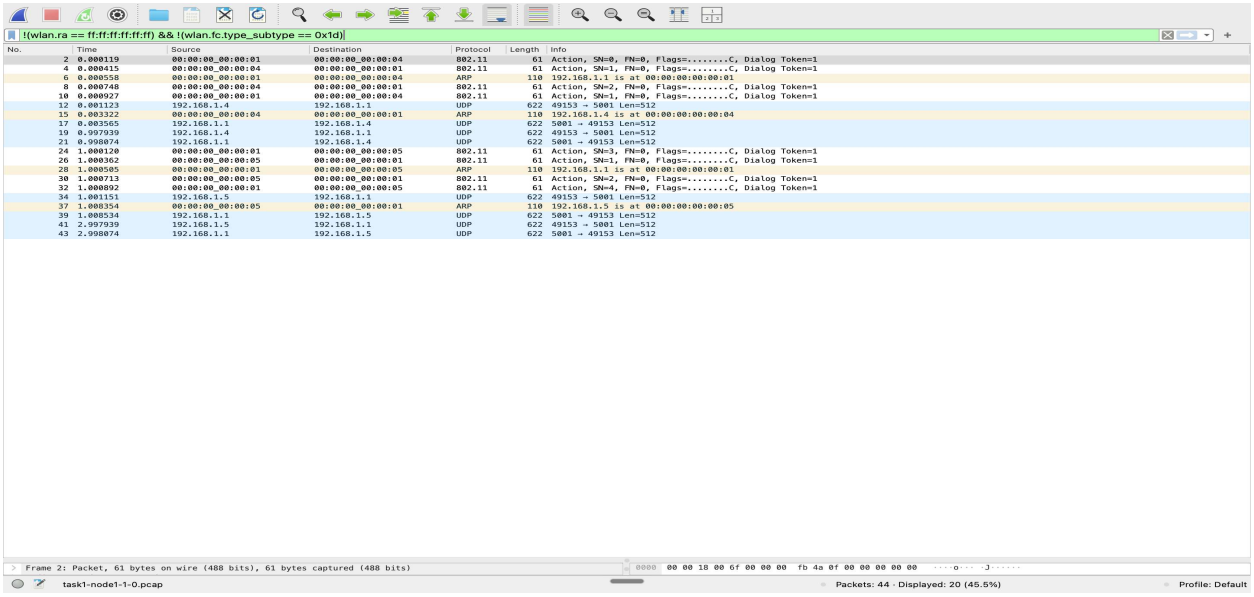


Figure 1d. Capture with filter !(wlan.ra == ff:ff:ff:ff:ff:ff) && !(wlan.fc.type_subtype == 0x1d): status bar shows 20 unicast non-ACK frames displayed.

- **Ans2**

  No collisions are visible in Node 1's capture. Every UDP data frame is immediately followed by its ACK frame. Filtering the capture with wlan.fc.retry == 1 confirms this directly: it returns 0 matching frames (Figure 2), so no frame was ever retransmitted. This is consistent with CSMA (carrier sensing and random back-off) combined with light traffic. Node 1 is neither the sender nor the receiver of any captured frame, so its capture only shows what it overhears.

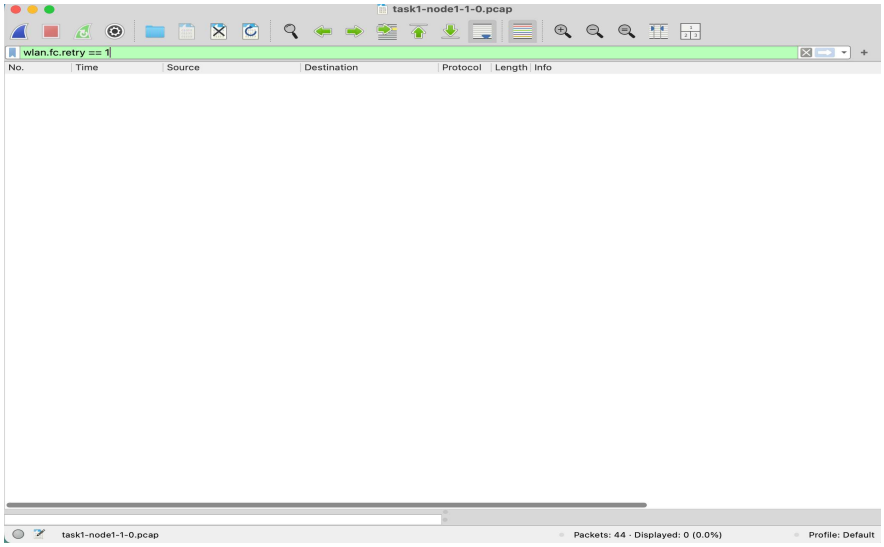


Figure 2. Capture at Node 1 with the filter wlan.fc.retry == 1: 0 frames displayed (0.0%).

3. Conclusions

We learned how to build our own network, write code in ns-3 and read frames in Wireshark. We built 5 Wi-Fi nodes in ad-hoc mode (802.11ac) with UDP echo traffic, a pcap on Node 1 and random-walk mobility. We started from third.cc and removed the point-to-point (wired link), CSMA (wired Ethernet) and AP (access point) parts.

Task 2

1. Experimental setup

Network Specifications:
physical (802.11ac, default channel), link (infrastructure mode: 5 stations with ns3::StaWifiMac and 1 AP with ns3::ApWifiMac, SSID EECE5155), network (IPv4, 192.168.2.0/24), transport (UDP), application (echo server and 2 clients), pcap on Node 4 and on the AP. RTS/CTS is not forced in the first run.

Code	Explanation
wifiStaNodes.Create(5) and wifiApNode.Create(1)	5 stations and 1 access point
wifi.SetStandard(WIFI_STANDARD_80211ac)	Default 802.11ac PHY parameters
YansWifiChannelHelper::Default() and YansWifiPhyHelper	Default wireless channel
mac.SetType("ns3::StaWifiMac", "Ssid", SsidValue(Ssid("EECE5155")))	Stations join the network named EECE5155
mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(Ssid("EECE5155")))	The AP advertises the SSID EECE5155 in its beacons
RandomWalk2dMobilityModel with Rectangle(-90, 90, -90, 90) on the stations	Stations move randomly inside a 180 m × 180 m area
ConstantPositionMobilityModel on the AP	The AP stays at a fixed position
address.SetBase("192.168.2.0", "255.255.255.0")	Puts all nodes in the 192.168.2.0/24 subnet
Ipv4GlobalRoutingHelper::PopulateRoutingTables()	Ideal routers exchanging routing tables in the background
UdpEchoServerHelper echoServer(5002) on Node 0	Server on Node 0, port 5002
Client on Node 3: Start(3.0), Interval(2.0) and MaxPackets(2)	Sends at 3 s and 5 s
Client on Node 4: Start(2.0), Interval(3.0) and MaxPackets(2)	Sends at 2 s and 5 s
PacketSize = 512	512-byte echo payload
SetPcapDataLinkType(DLT_IEEE802_11_RADIO) and EnablePcap(...) on Node 4 and on the AP	Captures only at Node 4 and the AP. The radiotap header lets Wireshark decode the A-MPDU frames.
Config::SetDefault("ns3::WifiRemoteStationManager::RtsCtsThreshold", UintegerValue(0)) – second run only	Forces the RTS/CTS handshake on every frame (Q4)

2. Results

- **Q1**
  From the very start of the simulation the AP sends beacon frames (frame 1 is the first one), a broadcast every 102.4 ms that advertises the network name "EECE5155". As each station turns on, it joins the network: it sends an Association Request, receives an ACK and then receives an Association Response from the AP. The stations also set up Block Ack sessions with the AP before sending data. Once the setup is done, the AP keeps sending beacons continuously in the background. Note: the Time column in our Wireshark figures is relative to the first captured frame, which is this first beacon. The beacon's own timestamp field shows it was sent about 0.1 s after the start of the simulation, so simulation times are about 0.1 s later than the times shown in the figures.

  Frames present here but absent in Task 1:
  - Beacon frames, with which the AP announces the network (no AP exists in ad-hoc mode).
  - Association Request/Response frames between each station and the AP.

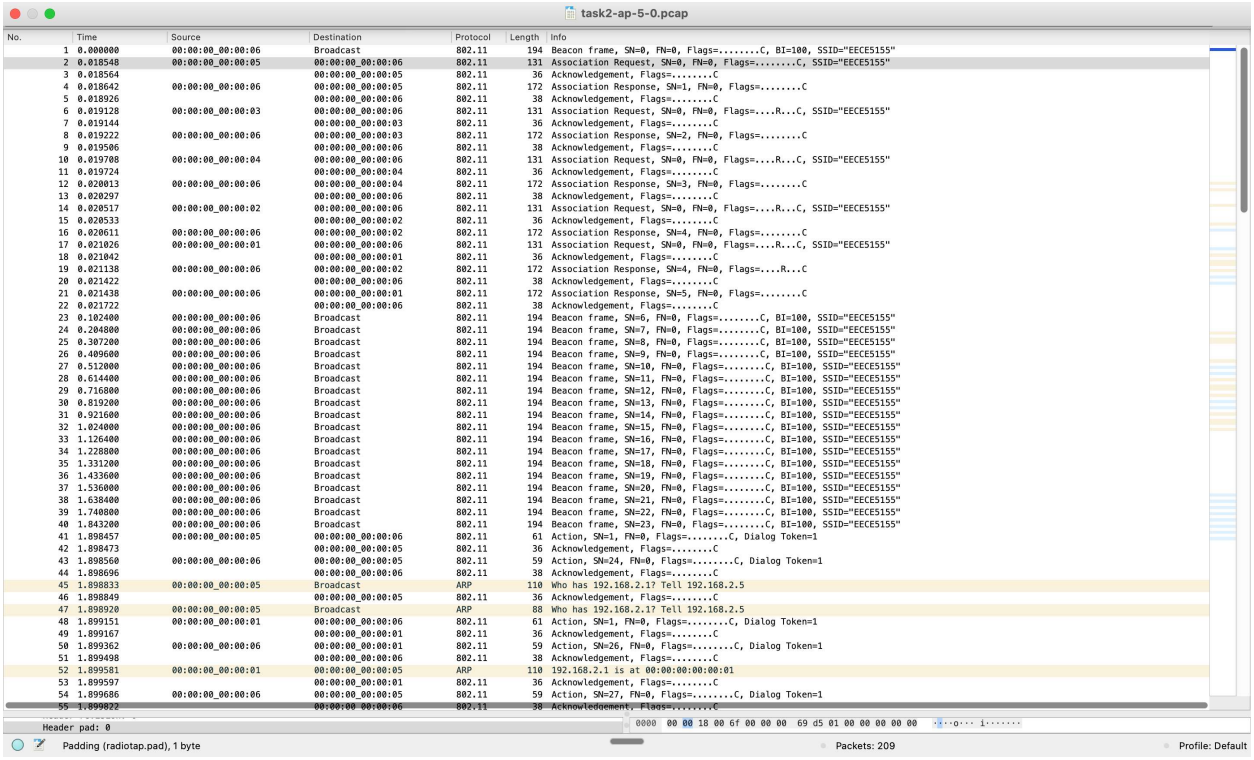


Figure 3. AP capture (first ~20 frames) showing the first beacon (frame 1), the association exchange and the Block Ack setup.

- **Q2**
  The most relevant parameters defined in the beacon frame are:
  - **SSID:** the network name, so stations know which network the beacon belongs to.
  - **Beacon Interval:** how often to expect the next beacon.
  - **Timestamp:** lets stations synchronize their clocks with the AP.
  - **Capability Information:** flags describing the BSS.
  - **Supported Rates:** the PHY data rates supported by the BSS.

- **HT/VHT Capabilities:** the 802.11n/ac features the AP supports (channel width, MCS set, spatial streams).

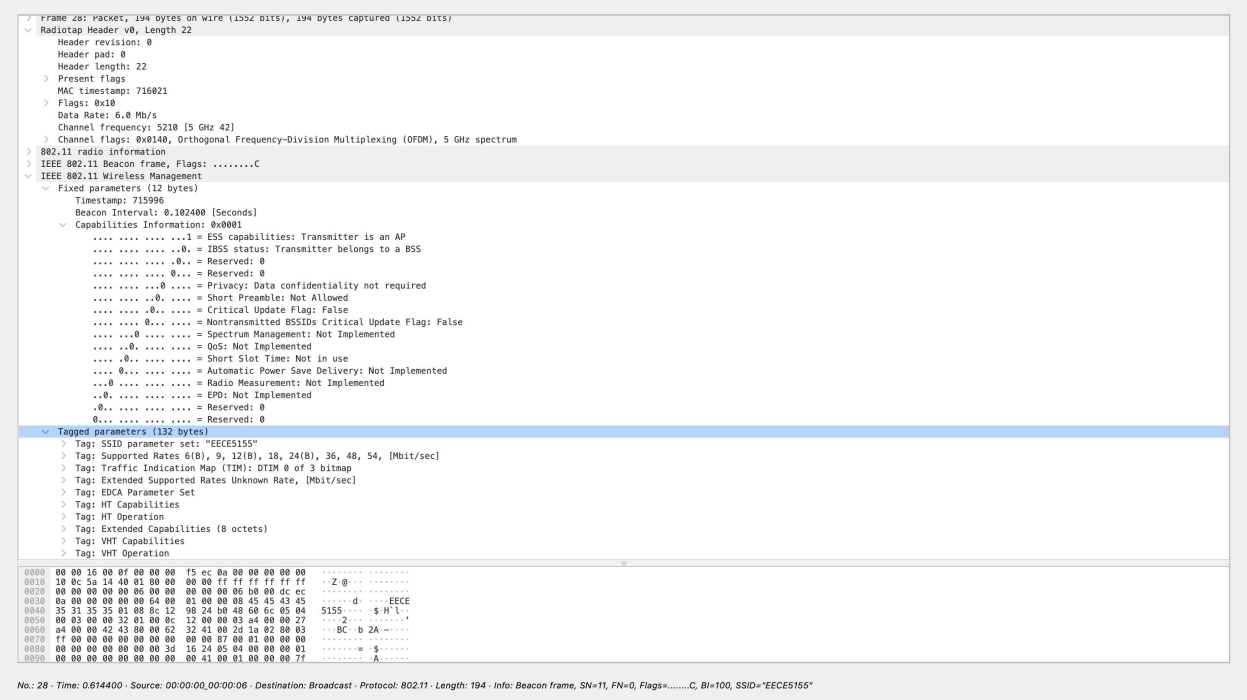


Figure 4. Expanded beacon frame (frame 28) in Wireshark showing the fields listed above.

- **Q3**
  Yes. The filter wlan.fc.retry == 1 returns 7 frames (Figures 5a and 5b), which fall into two very different categories:
  - At the very start of the simulation, several Association Request/Response frames are retried. All 5 stations try to associate with the AP almost simultaneously, so they contend for the channel.
  - Around t = 5 s (4.9 s in Wireshark's relative Time column), two UDP data frames are retried, one from Node 3 and one from Node 4, both destined to the server. This is the collision between the two clients that transmit at the same time.

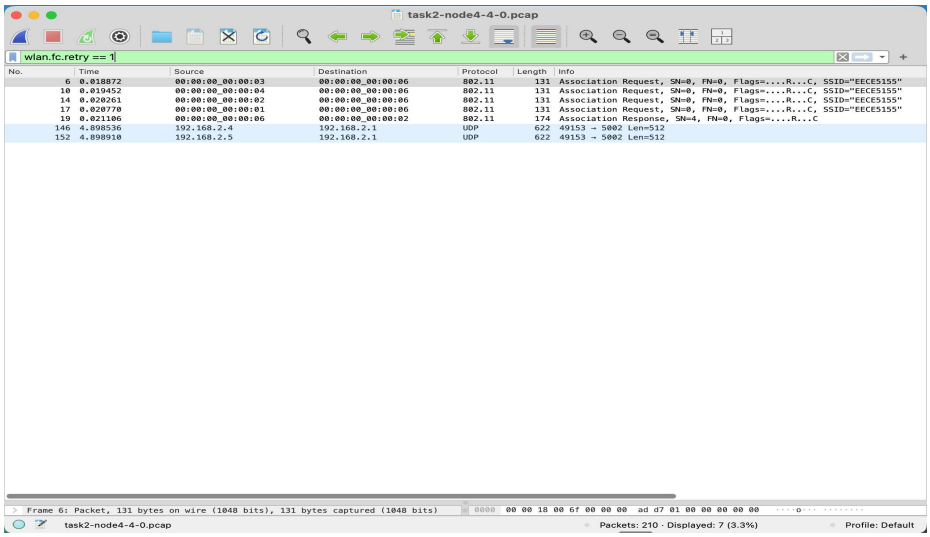


Figure 5a. Capture at Node 4 without RTS/CTS, filter wlan.fc.retry == 1: 7 frames displayed.

- **Q4**
  - **(a)**

    The nodes are forced to use RTS/CTS by setting the RtsCtsThreshold attribute of ns3::WifiRemoteStationManager:

    Config::SetDefault("ns3::WifiRemoteStationManager::RtsCtsThreshold", Uinteger-Value(0));

    The attribute sets the frame size above which ns-3 protects a transmission with an RTS/CTS handshake. Config::SetDefault applies the value to every station in the simulation, and it must be called before the devices are created so that the stations pick it up. We chose the value 0: our packets are only 512 bytes, so with a threshold of 0 every frame goes through the handshake and no frame can fall under the threshold and skip it. This gives complete coverage for observing the effect of RTS/CTS.

  - **(b)**

    No. With RTS/CTS enabled, the filter wlan.fc.retry == 1 && wlan.fc.type == 2 returns 0 data-frame retries (Figure 7), whereas the run without RTS/CTS had 2 retried UDP data frames (Figure 5b). Every data frame is now preceded by a short RTS/CTS exchange, which reserves the channel and sets the NAV at the other stations. So even when Node 3 and Node 4 transmit simultaneously (t = 5 s), a collision hits the short RTS frame instead of the whole data frame, and the data frame itself does not need to be retransmitted.

    The filter wlan.fc.type_subtype == 0x1b (Figure 6) displays 56 RTS frames out of 325 captured packets, which confirms that the handshake is active in this run.

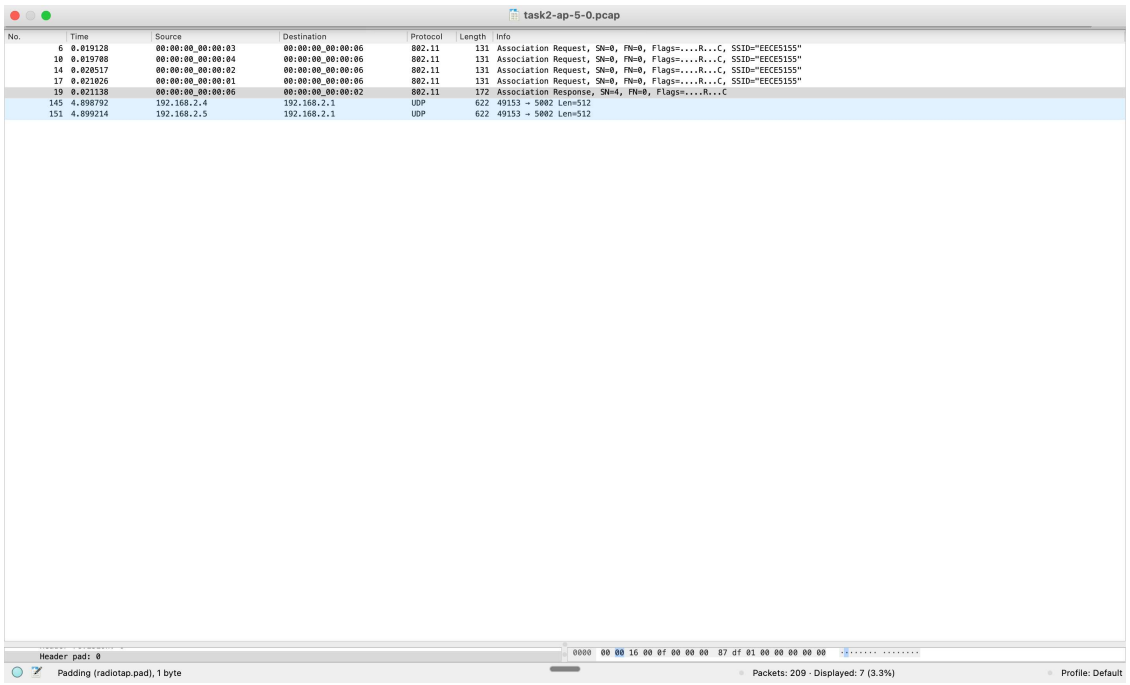


Figure 5b. Capture at the AP without RTS/CTS, filter wlan.fc.retry == 1: 7 frames displayed (2 UDP data-frame retries at t ≈ 5 s, shown as 4.9 s in Wireshark's relative Time column).

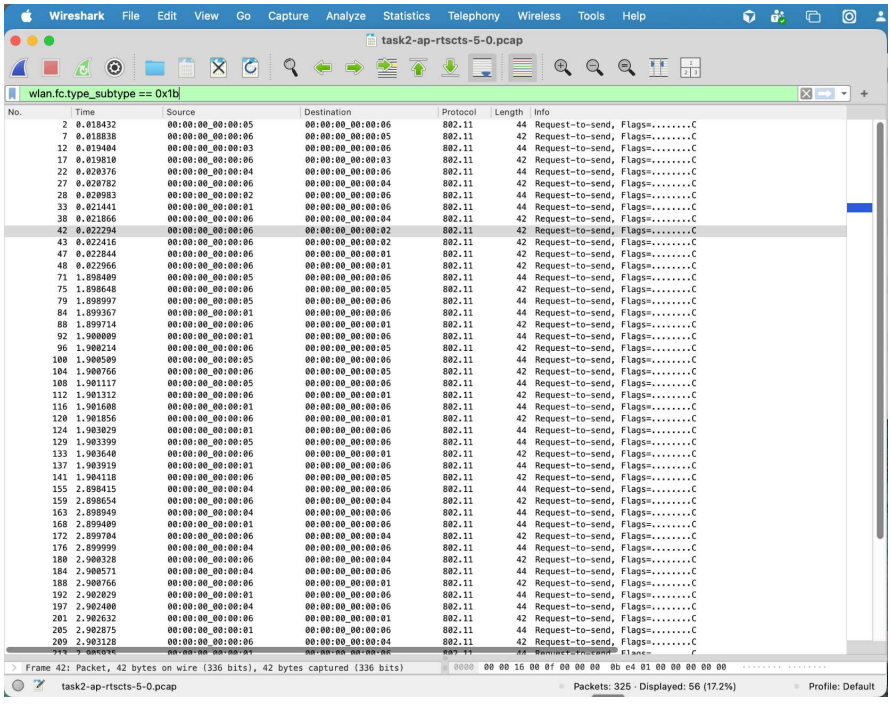


Figure 6. Capture at the AP with RTS/CTS enabled, filter wlan.fc.type_subtype == 0x1b (RTS frames): 56 of 325 packets displayed.

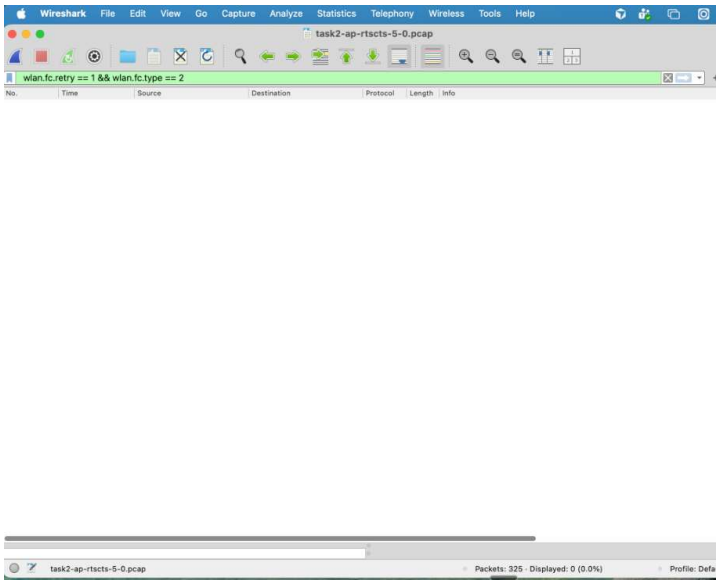


Figure 7. Capture at the AP with RTS/CTS enabled, filter wlan.fc.retry == 1 && wlan.fc.type == 2: 0 packets displayed.

- **(c)**

RTS/CTS reduces the cost of a collision: losing a short control frame is much cheaper than losing a whole data frame. It also solves the hidden-node problem, in which two stations cannot hear each other but can both reach the same AP.

How it works:
1. Before sending data, the sender transmits a short RTS (Request to Send) frame to the receiver.
2. The receiver replies with a CTS (Clear to Send) frame.
3. Both RTS and CTS carry a Duration field that says how long the channel will be occupied. Every other station that overhears these frames defers its transmission for that duration, including stations that cannot hear the original sender but can hear the receiver's CTS.
4. Only after receiving the CTS does the sender transmit the data frame, which the receiver acknowledges with an ACK.

- **(d)**

The NAV information is in the **Duration** field of the IEEE 802.11 header of the RTS and CTS frames. It tells the surrounding stations for how many microseconds they must defer channel access, i.e. how long to set their NAV timer. In Wireshark, select an RTS or CTS frame, expand "IEEE 802.11" and read the "Duration" field. In our capture, RTS frame No. 42 has Duration = 1600 µs (Figure 8).

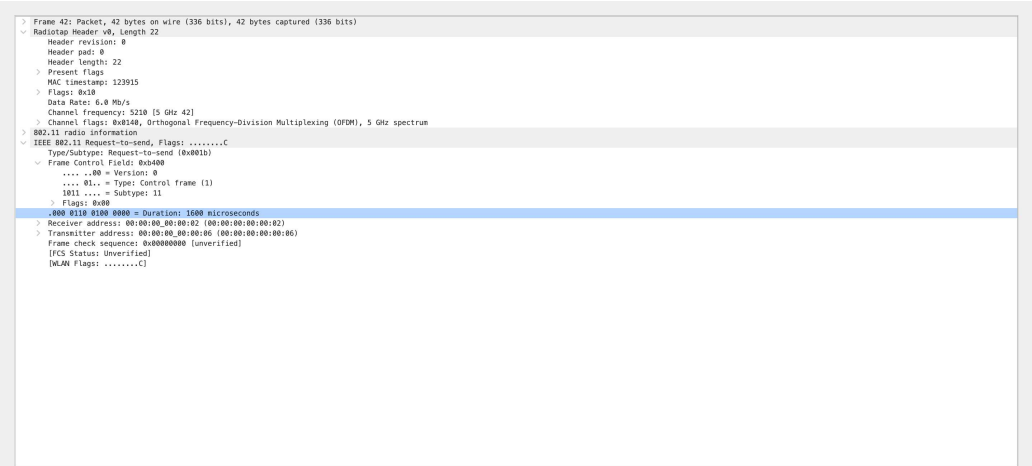


Figure 8. RTS frame (No. 42) with the IEEE 802.11 header expanded: Duration = 1600 µs.

- **(e)**

  No. In this scenario RTS/CTS slightly increases the end-to-end delay instead of improving it. Delay was measured as the time between the client sending the echo request and the client receiving the echo reply, taken from the console output timestamps.

Transmission	**Without RTS/CTS**	**With RTS/CTS**
T = 2 s (Node 4)	4.29 ms	5.88 ms
T = 3 s (Node 3)	3.30 ms	4.89 ms
T = 5 s (Node 4)	1.02 ms	1.44 ms
T = 5 s (Node 3)	1.46 ms	2.15 ms

Each transmission with RTS/CTS takes an additional 0.4–1.6 ms. Two extra frames (RTS and CTS) and their inter-frame gaps are sent in front of every frame, whether or not a collision would have happened. At the 6 Mb/s control rate visible in Figure 8, one RTS+CTS exchange lasts about 0.13 ms, and an echo round trip contains two of them (request and reply), i.e. about 0.26 ms. The T = 5 s rows, which carry no setup traffic, gain 0.42 ms and 0.69 ms, slightly more than this, because Node 3 and Node 4 also contend with each other and defer through back-off and the NAV. The T = 2 s and T = 3 s rows gain about 1.6 ms, most likely because the ARP and Block Ack setup frames sent before each client's first packet are also protected by an RTS/CTS exchange. Collision overhead is small in this scenario because the traffic is light (only 2 retries without RTS/CTS), so the handshake adds delay without saving anything.

3. Conclusions

Task 2 showed the difference between an infrastructure network and an ad-hoc Wi-Fi network. Unlike in Task 1, every station had to associate with the AP before exchanging data, and the AP sent periodic beacons to keep the network alive, which does not happen in ad-hoc mode. We also saw first-hand that RTS/CTS adds a small delay to every packet in exchange for protection against collisions. With the light traffic of this task there were few collisions, so RTS/CTS was not a good choice here. We expect this trade-off to change in a more congested network.

Task 3

1. Experimental setup

Network Specifications:
physical (802.11ac, channel 36, 20 MHz width, 20 dBm transmit power, ns3::MinstrelHtWifiManager), channel (ConstantSpeedPropagationDelayModel and FriisPropagationLossModel at 5.18 GHz), link (infrastructure mode: 1 AP and 1 STA), network (IPv4), transport (UDP), application (continuous UDP uplink from the STA to the AP, port 5003), pcap on the AP.

Code	Explanation
cmd.AddValue("distance", "STA-AP distance in m", distance)	The distance is a command-line argument, e.g. ./ns3 run 'lab1-task3-distance --distance=320', so the source never has to be edited between runs
ConstantPositionMobilityModel: AP at (0, 0, 1.5), STA at (distance, 0, 1.5)	Fixed positions; only the distance changes
Channel 36, 20 MHz width	5 GHz channel with 20 MHz bandwidth
TxPowerStart = TxPowerEnd = 20 dBm	Fixed transmit power
ns3::MinstrelHtWifiManager	Rate adaptation
ConstantSpeedPropagationDelayModel	Propagation delay
FriisPropagationLossModel with Frequency = 5.18e9	Free-space path loss at the carrier frequency of channel 36
UdpServerHelper on the AP (port 5003) and UdpClientHelper on the STA	Continuous UDP uplink flow from the STA to the AP
PacketSize = 1024, Interval = 81.92 µs, MaxPackets = 300000	About 100 Mbps offered load; the client never stops before the end of the simulation
Start(1.0), Stop(12.0) and Simulator::Stop(13.0)	Traffic from 1 s to 12 s; one extra second so packets in flight are delivered
RngSeedManager::SetSeed(5155) and SetRun(1)	Fixed random seed, so the results are reproducible
FlowMonitorHelper::InstallAll() before Run(); CheckForLostPackets() and GetFlowStats() after Run()	Throughput = received bytes × 8 / (last reception − first reception)
SetPcapDataLinkType(DLT_IEEE802_11_RADIO) and EnablePcap("task3-distance-<d>m", apDevice)	Radiotap capture on the AP; the distance is in the file name so runs do not overwrite each other

2. Results

- **Q1**

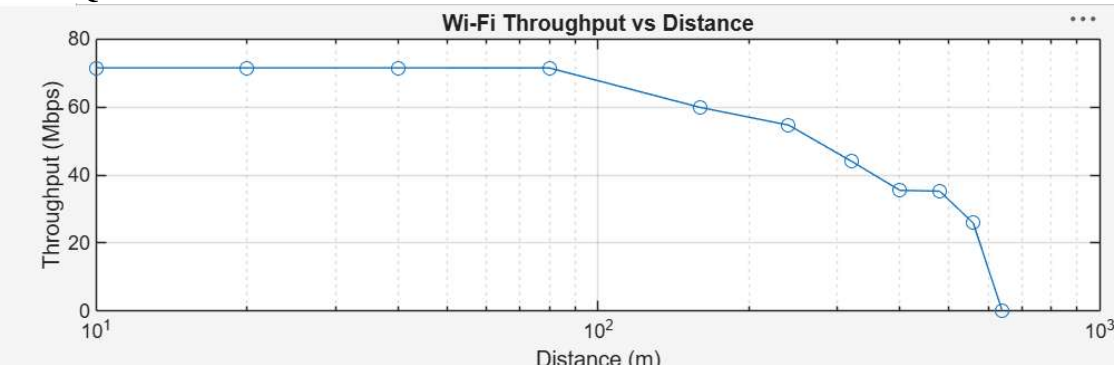


Figure 9. Throughput versus distance (logarithmic distance axis).

The throughput stays almost constant at about 71.47 Mbps from 10 m to 80 m. It starts to fall at 160 m, where the measured throughput is 59.86 Mbps, and keeps falling as the STA moves away from the AP: 54.66 Mbps at 240 m, 44.09 Mbps at 320 m, 35.50 Mbps at 400 m, 35.26 Mbps at 480 m and 25.91 Mbps at 560 m. At 640 m no packet is delivered, so the throughput is 0 Mbps. The throughput does not decrease gradually down to zero: it falls in steps with distance, and it stops abruptly between 560 m and 640 m.

At 640 m, the filter wlan.fc.tods == 1 && wlan.fc.type == 2 displays 0 frames (Figure 10), so no STA-to-AP data frames were captured at this distance.

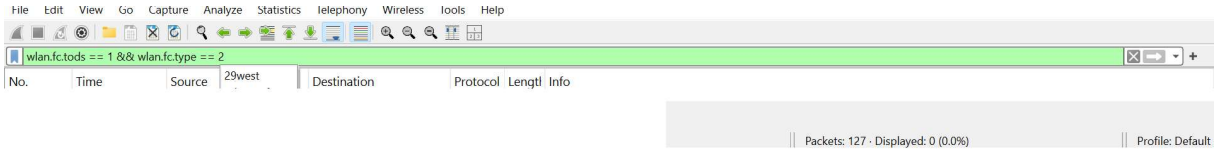


Figure 10. 640 m capture with the filter wlan.fc.tods == 1 && wlan.fc.type == 2: 0 frames displayed.

After removing the filter (Figure 11), the AP is still transmitting beacon frames with the SSID EECE5155, so the AP is still operating. The failure is therefore on the wireless link between the STA and the AP: at 640 m the STA is too far away for its data frames to be delivered.

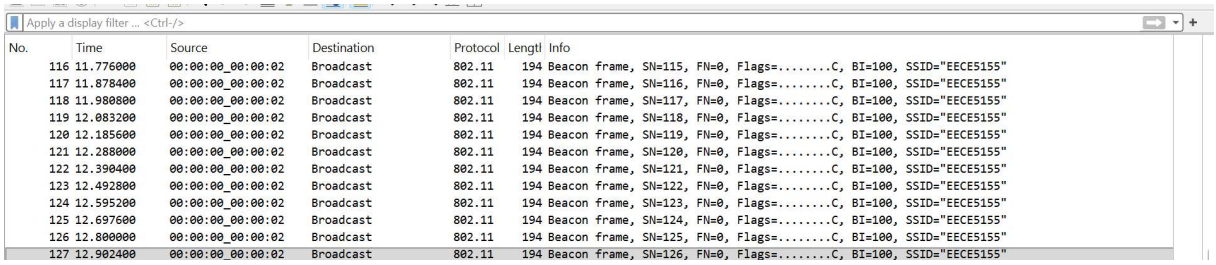


Figure 11. 640 m capture without the filter: the AP keeps sending beacon frames.

- **Q2**

The Friis free-space model gives the path loss and the received power (antenna gains of 0 dB, f = 5.18 × 10^9 Hz):

Loss (dB) = 20 log10(d) + 20 log10(f) − 147.55

Received power (dBm) = 20 dBm − Loss (dB)

Distance (m)	Path loss (dB)	Received power (dBm)	Measured throughput (Mbps)
10	66.74	−46.74	≈71.47
20	72.76	−52.76	≈71.47
40	78.78	−58.78	≈71.47
80	84.80	−64.80	≈71.47
160	90.82	−70.82	59.86
240	94.34	−74.34	54.66
320	96.84	−76.84	44.09
400	98.78	−78.78	35.50
480	100.36	−80.36	35.26
560	101.70	−81.70	25.91
640	102.86	−82.86	0

Every time the distance doubles, the received power drops by about 6.02 dB (20 log10 2). For example, it falls from −46.74 dBm at 10 m to −52.76 dBm at 20 m, and from −76.84 dBm at 320 m to −82.86 dBm at 640 m.

Relation to the plot: while the received power is high (−46.7 to −64.8 dBm, 10–80 m) the throughput is flat. Once the power falls below about −70 dBm (160 m and beyond), the link can no longer sustain its best rates and the throughput decreases. At 640 m the received power, −82.86 dBm, is just below the roughly −82 dBm minimum sensitivity of 802.11ac for the lowest rate on a 20 MHz channel, so no data packets are delivered. The Wireshark readings agree with the theory: about −47 dBm at 10 m and −77 dBm at 320 m, against −46.74 dBm and −76.84 dBm computed.

- **Q3**
  At 10 m, the selected STA-to-AP data frame has a PHY data rate of 39.0 Mb/s (MCS 4) and a signal strength of −47 dBm (Figure 12). Adding wlan.fc.retry == 1 to the filter displays 0 frames (Figure 13). At 320 m, the selected frame also shows a PHY data rate of 39.0 Mb/s, but the signal strength is only −77 dBm (Figure 14), and the retry filter displays 4655 retransmitted frames (Figure 15). The 320 m capture therefore has many more frames with the Retry flag set to 1.

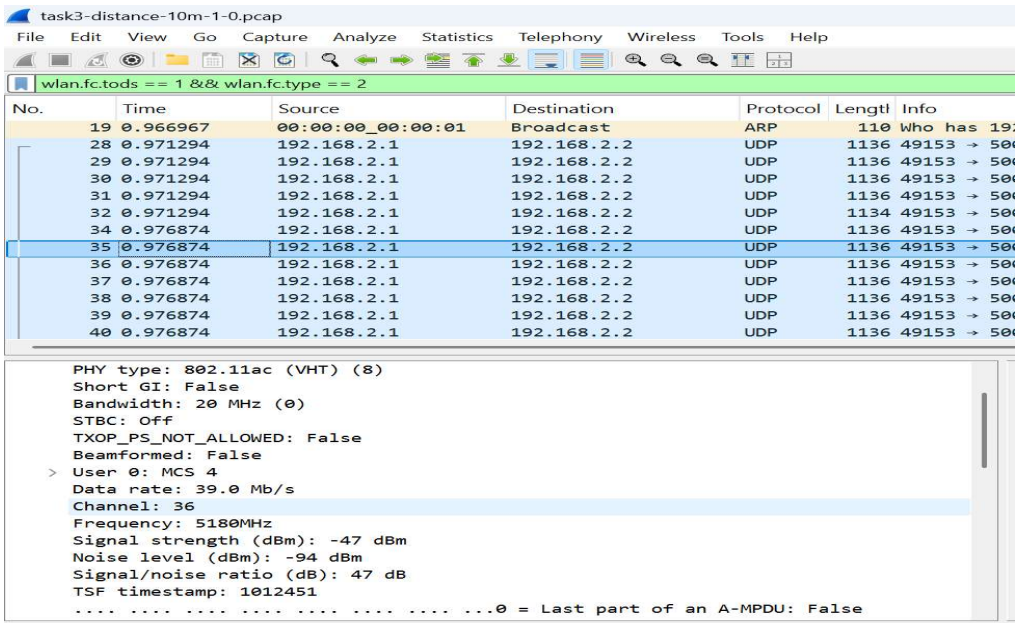


Figure 12. 10 m capture: STA-to-AP UDP data frame, PHY data rate 39.0 Mb/s, signal strength −47 dBm.

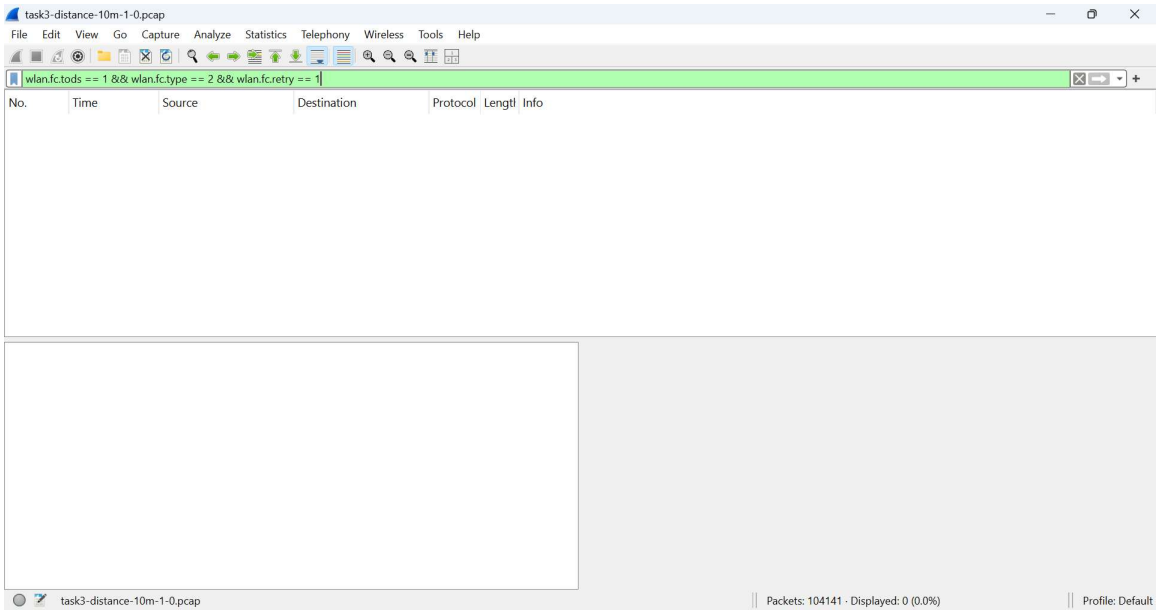


Figure 13. 10 m capture with wlan.fc.retry == 1 added to the filter: 0 frames displayed.

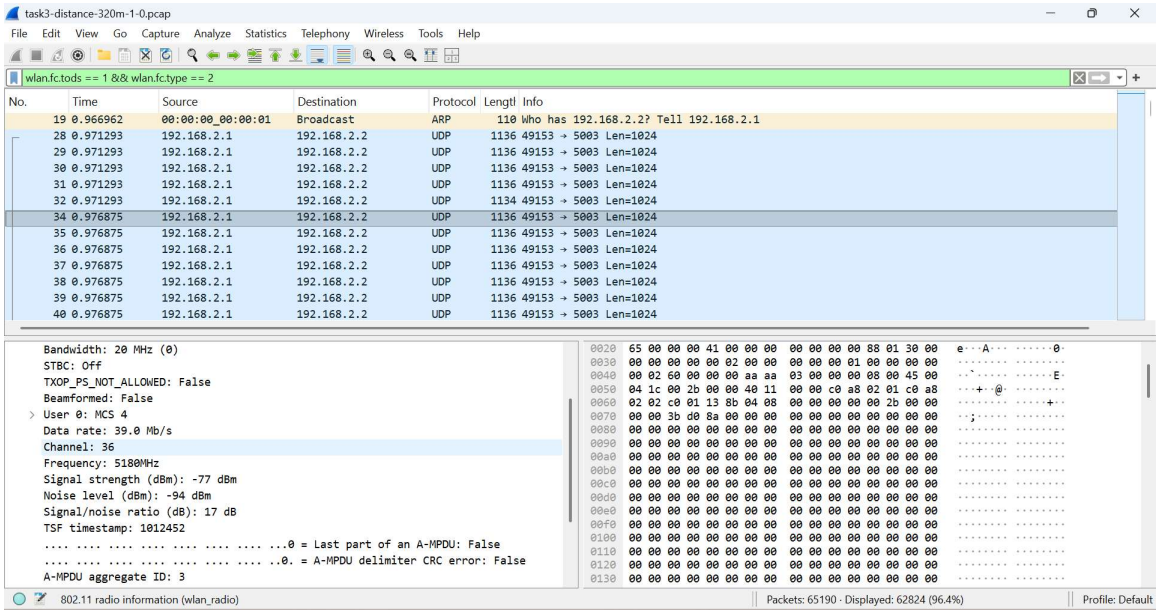


Figure 14. 320 m capture: STA-to-AP UDP data frame, PHY data rate 39.0 Mb/s, signal strength −77 dBm.

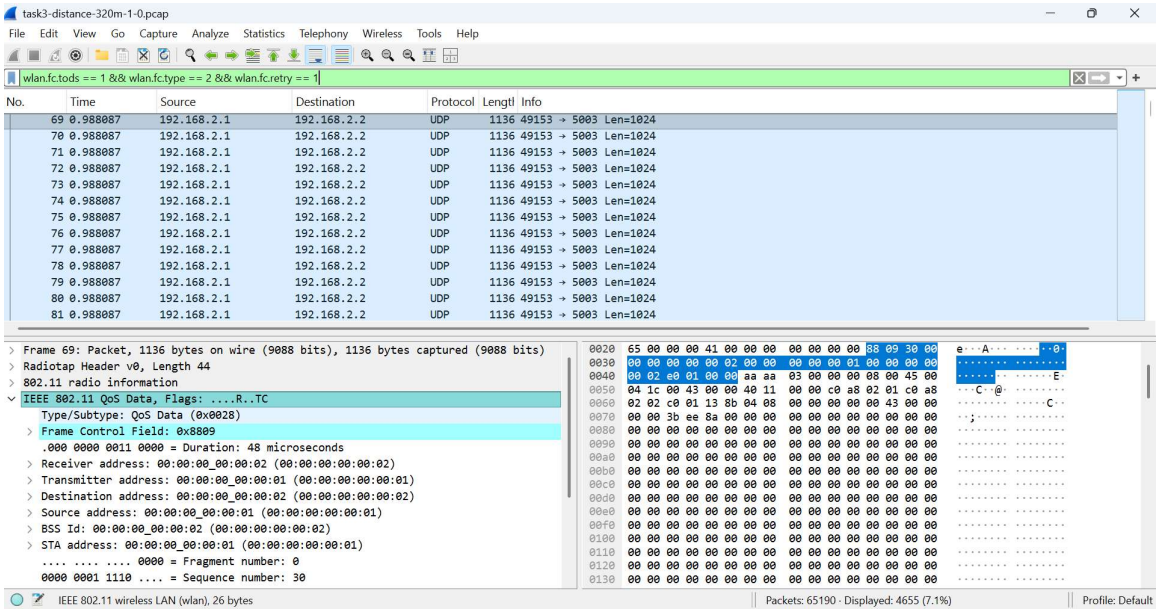


Figure 15. 320 m capture with wlan.fc.retry == 1 added to the filter: 4655 frames displayed.

Retries. At 320 m the received signal is 30 dB weaker than at 10 m (−77 dBm against −47 dBm), and the signal-to-noise ratio in the radiotap header drops from 47 dB to 17 dB. More frames are corrupted, are not acknowledged and have to be retransmitted, so the link is less reliable at 320 m.

PHY rate. The two inspected frames happen to have the same rate, 39.0 Mb/s. This is the rate of one individual frame: Minstrel adapts the rate over time, so a single frame does not describe the whole run. The 10 m throughput (71.47 Mbps) is higher than 39 Mb/s, so frames at higher MCS must also exist at 10 m. Inspect several frames, or filter on the radiotap data rate, and report the rate most frames use at each distance. The measured throughput is always

lower than the PHY rate because the channel also carries preambles, inter-frame spaces, back-off, headers, ACK/Block Ack frames and retransmissions, not only the UDP payload.

3. Conclusions

This task showed how distance affects the performance of a Wi-Fi link. The throughput stayed almost constant at short distances, started to drop after 80 m and fell to 0 at 640 m. The Friis calculations show that the received power decreases by about 6 dB every time the distance doubles. Wireshark showed many more retransmissions at 320 m than at 10 m, which confirms that the link is less reliable at 320 m. At 640 m the AP kept sending beacons, but the STA could not deliver data frames to the AP, so the break is in the STA-to-AP direction of the link.

Appendix: AI use

- **Tool and version:** Claude Sonnet 5.5
- **What I used it for:** understanding and troubleshooting our ns-3 code for Tasks 1,2.
- **What I verified or changed before including the output:** We ran the simulations ourselves and checked the frame counts, retries and beacon activity in our own Wireshark captures before including them in the report.

- **Tool and version:** ChatGPT
- **What I used it for:** interpreting the Task 3 instructions, troubleshooting parts of the ns-3 workflow, checking our Friis received-power calculations and clarifying the required report format.
- **What I verified or changed before including the output:** We ran the Task 3 simulations, wrote and ran the MATLAB code that plots our measured throughput, and checked the 10 m, 320 m and 640 m captures in Wireshark (frame counts, PHY data rates, signal strength, retries and beacon activity) before including them.